Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, general style, even when parts of it could be computed during compilation.

Original program

```mls
fun dot(xs, ys, i) =
  if xs.length == i then 0
  else xs.(i) * ys.(i)
    + dot(xs, ys, i + 1)

fun dotWith3(v) =
  dot([1, 0, 2], v, 0)
```

... can actually be written as

```mls
fun dotWith3(v) = v.(0) + 2 * v.(2)
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matching, the multiplication are all known at compile time. The residual program contains only the work that genuinely depends on the runtime input v.

Goal: let the programmer maintain the abstract version, and have the compiler automatically create a efficient implementation.

Monomorphism

Specialised, efficient versions of a function are created from a template function.

Rust

```rust
use std::ops::Mul;
fn f<T: Mul<Output=T>>(x: T, y: T) -> T {
  x * y
}

fn main() {
  // uses built-in *
  f(1, 2);
  f(1.3, 4.5);
}
```

Monomorphism

```rust
fn f1(x: i32, y: i32) -> i32 { x * y }
fn f2(x: f32, y: f32) -> f32 { x * y }

fn main() {
  // uses built-in *
  f1(1, 2);
  f2(1.3, 4.5);
}
```

Rust

Drawback: We are unable to specialise a function on the specific values on the arguments.

Rust allows for constant generics, which is limited.

Multi-Stage Programming

Well-known optimization technique, subfield of Partial Evaluation.

Example from [2]:

```
1  fun power(n, x) = if n is
2    0 then 1
3    else x * power(n - 1, x)
4  fun power2(x) = power(2, x)
```

Multi-Stage Programming

Well-known optimization technique, subfield of Partial Evaluation.

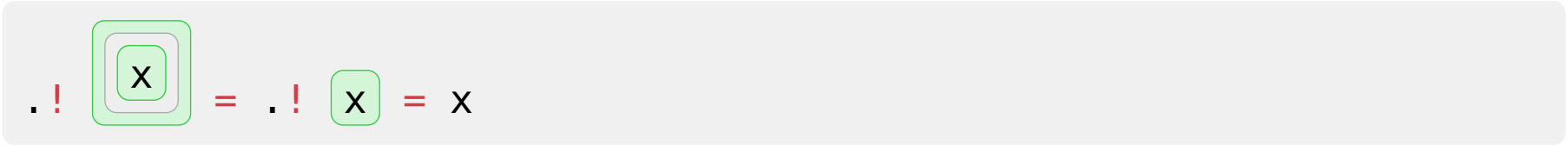
Example from [2]:

```
1  fun power(n, x) = if n is
2    0 then .<1>.
3    else .<.~x * .~(power(n - 1, x))>.
4  fun power2 = .! .<(x => .~(power(2, .<x>.)))>.
```

Here, green annotates code to be executed in the next stage, and gray executed in the current stage.

Multi-Stage Programming

Rule for `.!`: evaluate until we the whole code fragment is in the next stage, then move it to the current stage.



Analogous to `eval`:

```python
1  eval("eval(\"1\")") = eval("1") = 1
```

Python

During evaluation, we able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

```
1  x => power(2, x)                                                    mls
2  x => if 2 is 0 then 1 else x * power(2 - 1, x)
3  x => x * power(1, x)
4  x => x * if 1 is 0 then 1 else x * power(1 - 1, x)
5  x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)
6  x => x * x * 1
7  fun power2 = x => x * x * 1
```

Multi-Stage Programming

Drawback: The annotations need to be manually added, and so the library designer needs to anticipate uses of the library.

Drawback: Code fragments are evaluated separately, when the result of a code fragment may be reused.

```
1  fun power10 = .! .<( x => .~( power(10, .<x>.) ) )>.
```

mls

See memoization of code fragments in [3].

Hybrid Partial Evaluation [1]

```
1  class D1(val x)
2  class D2(val x)
3  let y = if x is
4    1 then new D1(1)
5    2 then new D2(2)
6  y.f()
```

Limitation: Hybrid Partial Evaluation lacks **disjunctive class shapes** (e.g., ⟦`D1(..)` ∪ `D2(..)`⟧), so `y.f()` cannot be specialized.

High-Level Approach

Start with an ordinary module:

```mls
module M with
  fun dot(xs, ys, i) =
    if xs.length == i then 0
    else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

High-Level Approach

Users can mark a module as `staged` which turns the module into a **code generator**. When we execute the program, it emits a new residual module instead of executing the program.

```mls
staged module M with
  fun dot(xs, ys, i) =
    if xs.length == i then 0
    else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

High-Level Approach

The output of executing the program is

```mls
module M with
  fun dotWith3(v) = v.(0) + 2 * v.(2)
  // ... with more specialised functions ...
```

which the user can import.

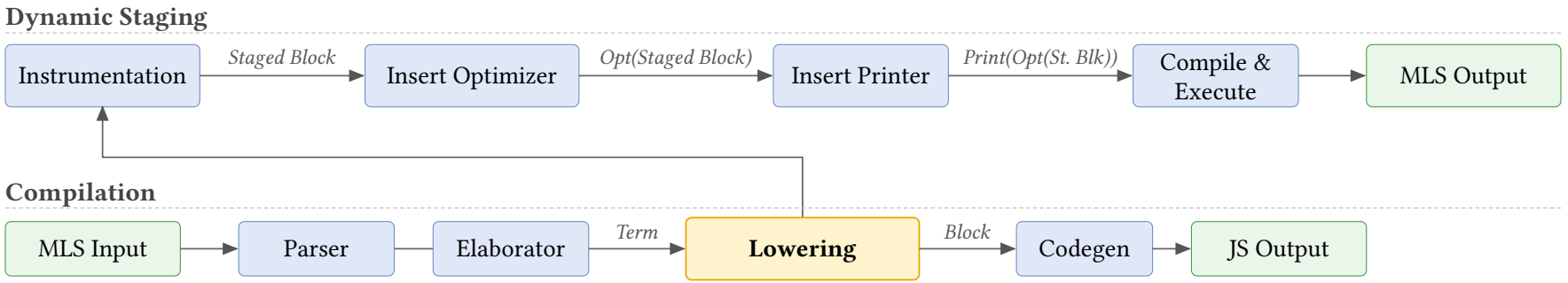
Hence, our approach is a combination of **metaprogramming** (code generator) and **specialization** (generation of specialised functions).

MLscript Compiler

MLscript Compiler is a compiler written in Scala that converts MLscript to JavaScript.

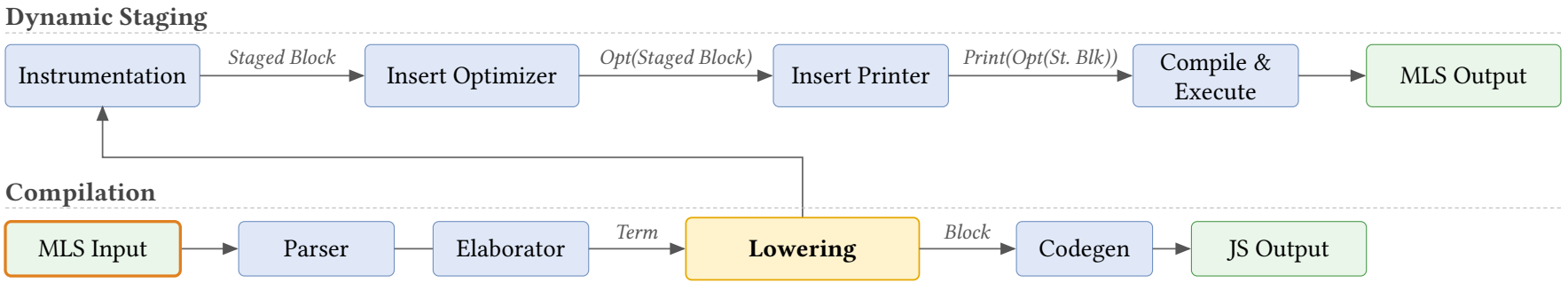
- Lexer/Parser: converts source code into AST
- Elaborator/Resolver: deal with references
- **Lowering**: converts AST of **Term** to AST of **Block**
- Codegen: turn AST into executable code (JavaScript).

Dynamic Staging Recipe



Two phases: Instrumentation / Execute Optimizer

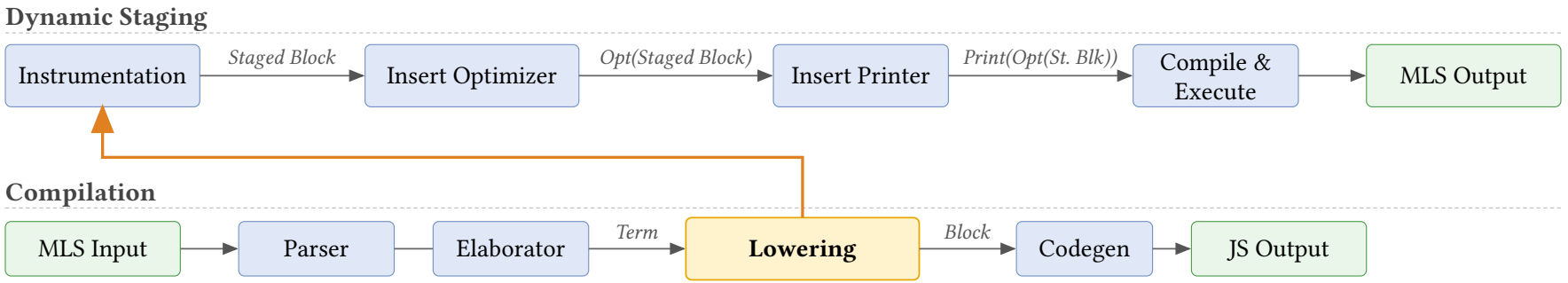
Dynamic Staging Recipe



Source (MLscript)

```mls
if C(0) is
  Int  then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

Dynamic Staging Recipe



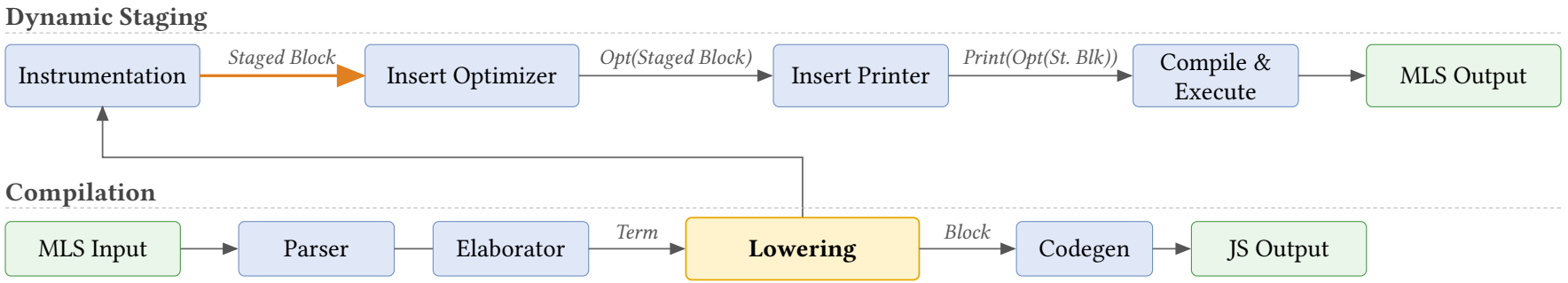
Source (MLscript)

```mls
if C(0) is
  Int  then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

Scala Block representation

```scala
Scoped(Set(scrut, n, arg, tmp),
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n,   Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ")"]))
  ], Ret("Unknown")))
```

Dynamic Staging Recipe



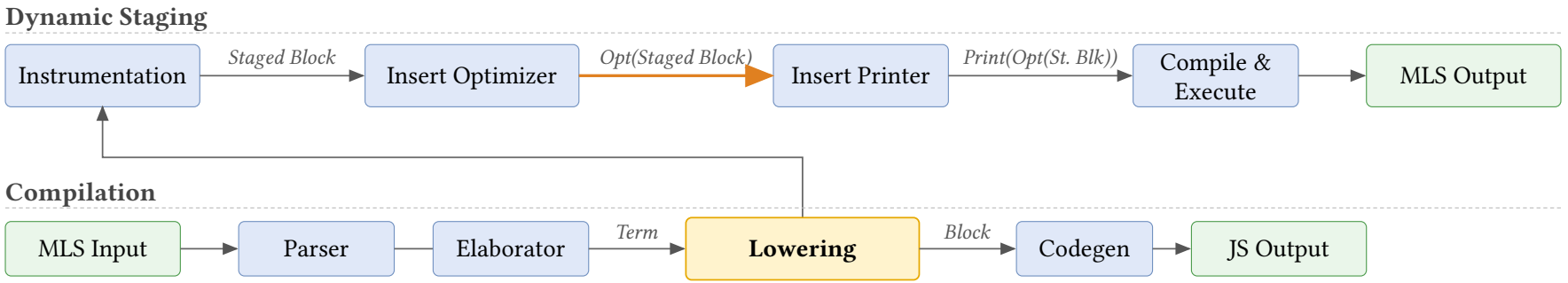
Staged Block

```
let b = Scoped([scrut, n, arg, tmp],                    mls
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n,   Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ")"]))
  ], Ret("Unknown")))
```

Scala Block representation

```
Assign(Symbol("b"),                                     Scala
  Call("Scoped", [
    Tuple(Ref(Symbol("scrut")), ...),
    Assign(Symbol("tmp"),
      Call("Assign", [Symbol("scrut")],
      ...
    )
  ]), ...
)
```

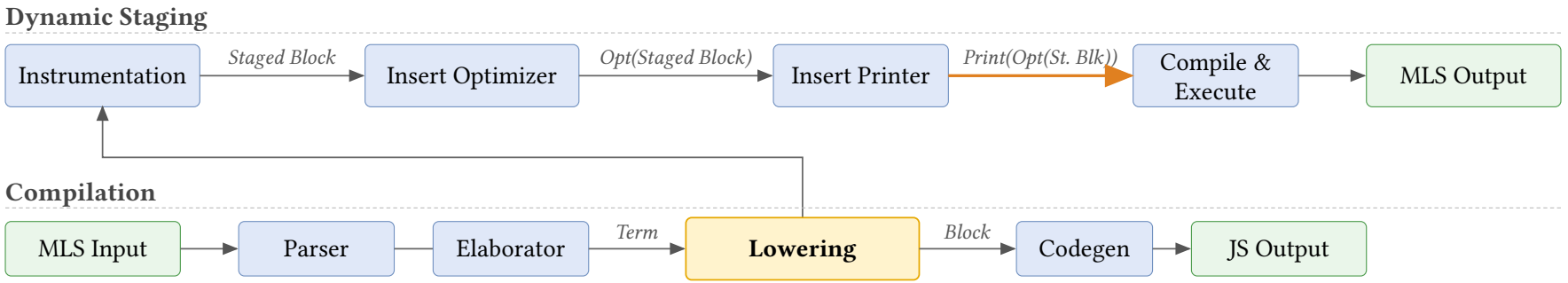
Dynamic Staging Recipe



Opt(Staged Block)

```
let b = ...                    mls
gen(b)
```

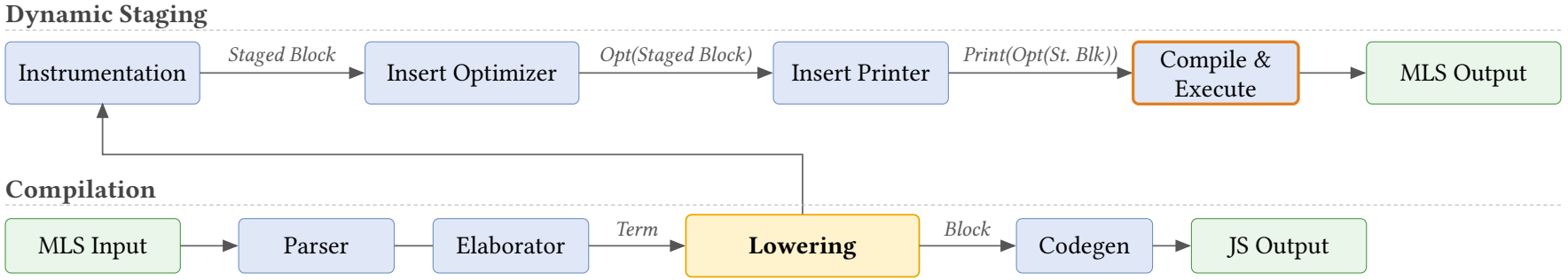
Dynamic Staging Recipe



Print(Opt(Staged Block))

```
let b = ...                                    mls
print(gen(b))
```

Dynamic Staging Recipe

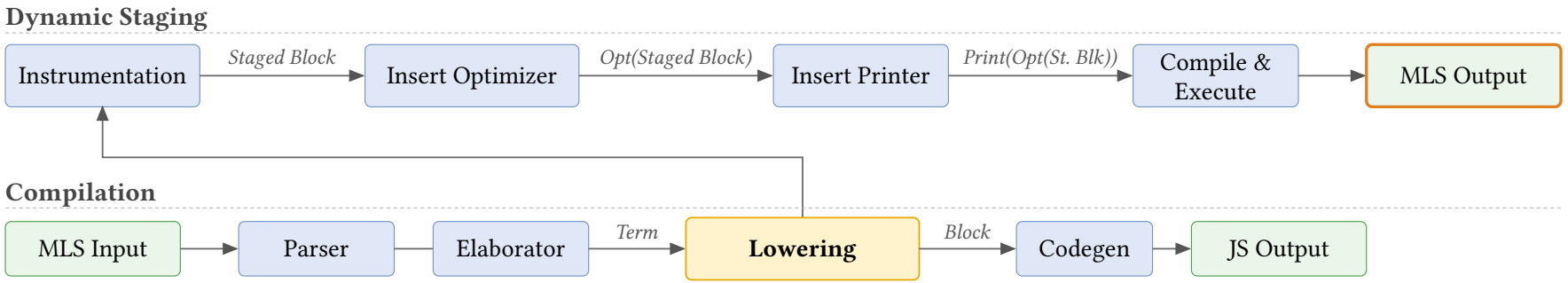


Compile & Execute

This execution is the **first stage** of metaprogramming. It entails the execution of

- the **optimizer** which outputs the **optimized staged block**,
- and the **printer** which prints the new MLscript program from the optimized staged block.

Dynamic Staging Recipe



Source MLscript

```mls
if C(0) is
  Int  then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

Generated MLscript

```mls
"C(0)"
```

Staging Block

```scala
case class Return(res, implct) extends Block
case class Match(scrut, arms, dflt, rest) extends Block
case class Assign(lhs, rhs, rest) extends Block
```

Scala

```mls
class Block with
  constructor
    Return(res, implct)
    Match(scrut, arms, dflt, rest)
    Assign(lhs, rhs, rest)
```

mls

For any Scala Block data, we can recreate the same structure within MLscript which we called **Staged Block**.

Reduce the coupling of shape propagation logic to Scala Block.

Staging Block

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

We perform structural induction to stage each type of Scala Block.

```
Value.Lit(true)
```

```
ValueLit(true)                    mls
```

```
Symbol("x")
```

```
Symbol("x")                       mls
```

Example:

- `ClassSymbol("C")` $\mapsto$ ...?

This is not enough for staging symbols. We'll revisit this case later on.

Staging Block

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

We perform structural induction to stage each type of Scala Block.

```
Value.Lit(true)
```

```
ValueLit(true)
```
mls

```
Value.Ref(Symbol("x"))
```

```
let l = Symbol("x")
ValueRef(l)
```
mls

```
Select(Value.Ref(Symbol("x")), Tree.Ident("p"))
```

```
let q = Value.Ref(Symbol("x"))
let n = Tree.Ident("p")
Select(q, n)
```
mls

Staging Block

```
Tuple([x0, x1, ...])
```

```
let s0 = x0                                                    mls
let s1 = x1
                                    ...
Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging Symbols

- Keep track of current value in Staged Block

  Add reference to current value in the symbol

```
Select(Value.Ref(ClassSymbol("C")), Tree.Ident("x"))
```

```mls
let CSym = ClassSymbol("C", C)
Select(ValueRef(CSym), Symbol("x"))
```

  Add redirection within current stage to allow access for next stage

```mls
staged class A with
  fun f() = M.f()
  val redirect_M = M
```

Staging Symbols

- Uniqueness of Symbols

  We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For local symbols, we can maintain a map during staging to reuse a staged symbol.

```mls
x + x
```

```mls
let x = Symbol("x")
// let x1 = Symbol("x")
Call(ValueRef(Symbol("+")), [[ValueRef(x), ValueRef(x)]])
```

Staging Symbols

- Uniqueness of Symbols

  We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For class and module symbols, we need to cache and use first instance of a symbol within the current runtime.

```
M.f()
```

```
1  let MSym = symbolMap.check(ModuleSymbol("M", M))          mls
2  Call(Select(ValueRef(MSym), Symbol("f")), [[]])
```

A short demo on Instrumentation

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

```mls
staged module Math with
  val funCache = new Map()
  val generatorMap = new Map([["pow", pow_gen]])
  fun pow_staged() = ...
  fun pow_gen(x, n) =
    specialise(funCache, "f", pow_staged, [x, n])
  fun propagate() =
    let dyn = ...
    pow_gen(dyn, dyn)
    sq_gen(dyn)
```

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

```mls
staged module Math with
  val funCache // memoize specialised functions
  val generatorMap // point to generator function
  fun pow_staged() // return staged version of the function
  fun pow_gen(x, n) // redirect to shape propagation
  fun propagate() // generates all entry functions
```

Instrumentation

For staged classes, we can treat the parameters of the class as the function parameter, then specialise the functions as usual.

```mls
staged class C(x, y) with
  fun add() = x + y
```

```mls
staged module C with
  fun add(cls)() = cls.x + cls.y
```

Function-to-Class Transformation

Convert the lambda functions into classes, so that they can be uniformly treated as classes.

```mls
fun f(y) = x => x + y

class Function(y) with
  fun call(x) = x + y
fun f(y) = Function(y)
```

Block in BNF

Symbol x, y, z *Literal* ι

Function definition $\mathbf{fun}\ f(\overline{x_i}^n) = b$ *Plain class definition* $\mathbf{class}\ C\left(\overline{\mathbf{val}\ n}\right)$

Staged module definition $\mathbf{staged\ module}\ M\ \mathbf{with\ fun}\ f(\overline{x_i}^n) = b$

$$
\begin{array}{rrcl}
\textit{Path} & p & ::= & \iota \mid x \mid p.n \\
\textit{Result} & r & ::= & p \mid f(\overline{p}) \mid \mathbf{new}\ C(\overline{p}) \mid [\overline{p}] \\
\textit{Match pattern} & \pi & ::= & \iota \mid C \mid []^n \mid \_ \\
\textit{Match arm} & \kappa & ::= & \pi\ \mathbf{then}\ b \\
\textit{Non-tail block} & B & ::= & \varepsilon \mid \mathbf{if}\ p\ \mathbf{is}\ \overline{\kappa}; B \mid x = r; B \mid \mathbf{let}\ x; B \\
\textit{Block} & b & ::= & \mathbf{return}\ r \mid \mathbf{end} \mid B; b
\end{array}
$$

Block Example

```mls
fun f(x) =
  let y = x.n
  if x is
    C then y + 1
    else new C(1)
```

Block Example

```mls
fun f(x) =
  let y = x.n
  if x is
    C then y + 1
    else new C(1)
```

Path $p ::= \iota \mid x \mid p.n$

references like `x` and field accesses `x.n`

Block Example

```
1  fun f(x) =                    mls
2    let y = x.n
3    if x is
4      C then y + 1
5      else new C(1)
```

Result r ::=

$p \mid f(\overline{p}) \mid \textbf{new}\ C(\overline{p}) \mid [\overline{p}]$

a path, call, `new`, or tuple

Block Example

```mls
1  fun f(x) =
2    let y = x.n
3    if x is
4      C then y + 1
5      else new C(1)
```

Match pattern $\pi ::= \iota \mid C \mid []^n \mid \_$

here: C (class) and _ (else)

Block Example

```
1  fun f(x) =                    mls
2    let y = x.n
3    if x is
4      C then y + 1
5      else new C(1)
```

Match arm $\kappa ::= \pi$ **then** b

a pattern paired with its body

Block Example

```mls
1  fun f(x) =
2    let y = x.n
3    if x is
4      C then y + 1
5      else new C(1)
```

Non-tail block B ::=

$\varepsilon \mid \textbf{if}\ p\ \textbf{is}\ \overline{\kappa}; B \mid x = r; B \mid \textbf{let}\ x; B$

Two non-tailed block: assignment and pattern matching

Block Example

```mls
fun f(x) =
  let y = x.n
  if x is
    C then y + 1
    else new C(1)
```

Block $b ::= \mathbf{return}\ r \mid \mathbf{end} \mid B; b$

There is an implicit End at this program since there is no return.

Shape Definition

A **Shape** captures what we statically know about a value at compile time.

$$\text{Shape } s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \bot \mid s \cup s$$

We write ⟦s⟧ to denote the shape of an expression, distinguishing it from actual values.

```
1  fun f(p, cond) =                                  mls
2    let a = 42                                      ⟦42⟧
3    let b = C(p, 2)                                 ⟦C(dyn, 2)⟧
4    let c = if cond then [1, 2] else C(1, 2)        ⟦[1,2] ∪ C(1,2)⟧
5    let d = if b is C then 0 else 1                 ⟦0⟧
```

The mechanism to calculate the shapes are called **Shape Propagation**.

Shape Definition

A Tiny Example

```mls
staged module M with
  fun add1(x) = x + 1
  fun test()  = add1(2)
```

Walking through `test()`:

1. Argument `x` has shape $[\![2]\!]$
2. Specialise `add1` with $\texttt{x} \mapsto [\![2]\!]$ inside the body, `x` looks up $[\![2]\!]$ in the context.
3. `x + 1` folds: $[\![2]\!] + [\![1]\!] \to [\![3]\!]$.
4. Cache the result as `add1_Lit2() = 3`; rewrite the call site `add1(2)` to `add1_Lit2()`.

Final shape of `test()`: $[\![3]\!]$. Body collapses to a constant.

Shape Definition

A Tiny Example (Residual Module)

The output residual module is generated with the specialized functions:

```mls
module M with
  fun add1_Lit2() = 3
  fun test() = add1_Lit2()
```

Now, we begin to explain the mechanism of Shape Propagation, which is broken down into **Shape of Path**, **Shape of Result**, then **Shape of Block**.

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

We maintain a context Γ which tracks the shape of each path encountered so that we can evaluate the shape of every path.

$$\frac{}{\Gamma \vdash \iota \Rightarrow [\![\iota]\!]}$$

Example

```mls
42
```

$\Gamma = \varepsilon$
$\varepsilon \vdash 42 \Rightarrow [\![42]\!]$

$$\frac{(p \mapsto s) \in \Gamma \quad s \neq \bot}{\Gamma \vdash p \Rightarrow s}$$

Example

```mls
x
```

$\Gamma = x \mapsto [\![C(n : 1)]\!]$
$\Gamma \vdash x \Rightarrow [\![C(n : 1)]\!]$

$$\frac{p \notin \mathrm{dom}(\Gamma) \quad \Gamma \vdash p \Rightarrow s}{\Gamma \vdash p.n \Rightarrow \mathrm{sel}(s, [\![n]\!])}$$

Example

```mls
x.n
```

$\Gamma = x \mapsto [\![C(n : 1)]\!]$
$\Gamma \vdash x.n \Rightarrow [\![1]\!]$

$$p ::= \iota \mid x \mid p.n$$

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

Shape Selection (`sel`)

sel computes the shape of a selection.

$$\mathrm{sel}([\![C(\overline{n_i : s_i})]\!], [\![n_i]\!]) = s_i \qquad \mathrm{sel}(\mathbf{dyn}, [\![n_i]\!]) = \mathbf{dyn}$$

$$\mathrm{sel}([\![[\overline{s_i}^{i \in 0..n}]]\!], [\![i]\!]) = s_i \qquad \mathrm{sel}(s_1 \cup s_2, s) = \mathrm{sel}(s_1, s) \cup \mathrm{sel}(s_2, s)$$

$$\mathrm{sel}(s_1, s_2) = \mathbf{err}$$

Example

$$\begin{aligned}\mathrm{sel}([\![1,2]\!] \cup [\![2,3,4]\!], [\![1]\!]) &= \mathrm{sel}([\![1,2]\!], [\![1]\!]) \cup \mathrm{sel}([\![2,3,4]\!], [\![1]\!]) \\ &= [\![2]\!] \cup [\![3]\!]\end{aligned}$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

We optimize block and track shapes simultaneously: r becomes r' with shape s.

$$\frac{\Gamma \vdash p \Rightarrow s}{\Gamma \vdash p \rightsquigarrow p \Rightarrow s}$$

Example

```
x                                    mls
```

$\Gamma = x \mapsto [\![1]\!]$
$\Gamma \vdash x \rightsquigarrow x \Rightarrow [\![1]\!]$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash [\overline{p_i}^n] \rightsquigarrow [\overline{p_i}^n] \Rightarrow [\![\overline{s_i}^n]\!]}$$

Example

```
[x, y]                               mls
```

$\Gamma = x \mapsto [\![1]\!], y \mapsto [\![2]\!]$
$\Gamma \vdash [x, y] \rightsquigarrow [x, y] \Rightarrow [\![1, 2]\!]$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash \mathbf{new}\ C(\overline{p_i}) \rightsquigarrow \mathbf{new}\ C(\overline{p_i}) \Rightarrow [\![C(\overline{n_i : s_i})]\!]}$$

Example

```
new Pair(x, y)                       mls
```

$\Gamma = x \mapsto [\![1]\!], y \mapsto [\![2]\!]$
$\Gamma \vdash \mathbf{new}\ \mathrm{Pair}(x, y) \Rightarrow [\![\mathrm{Pair}(a : 1, b : 2)]\!]$

$$r ::= p \mid [\overline{p}] \mid \mathbf{new}\ C(\overline{p}) \mid f(\overline{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) — Staged Application

When f is **staged**, we recursively specialise its body on the argument shapes.

$$\frac{f \quad \text{is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}\,(\overline{p}, \overline{s}) = (r', s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r' \Rightarrow s}$$

Example

```
staged module Staged with                                    mls
  fun pyth(x, y) = x * x + y * y
  fun test() = pyth(2, 3)
```

Specialise $\text{pyth}_{\text{gen}}\,([\![2]\!], [\![3]\!]) \rightarrow$ produces `pyth_Lit2_Lit3() = 13`, shape $[\![13]\!]$. The f_{gen} function will save the specialised function in a cache.

Conclusion: $\varepsilon \vdash \text{pyth}(2, 3) \rightsquigarrow \text{pyth\_Lit2\_Lit3}() \Rightarrow [\![13]\!]$

$$r ::= p \mid [\overline{p}] \mid \textbf{new}\, C(\overline{p}) \mid f(\overline{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) — Staged Application

After propagation, the call site `pyth(2, 3)` is rewritten to its cached variant.

Example

```mls
staged module Staged with
  fun pyth_Lit2_Lit3() = 13
  fun test() = pyth_Lit2_Lit3()
```

Cache

Function	Shapes	Symbol	Block	Returned Shape
`pyth`	$(\llbracket 2 \rrbracket, \llbracket 3 \rrbracket)$	`pyth_Lit2_Lit3`	`return 13`	$\llbracket 13 \rrbracket$
`pyth`	$(\mathbf{dyn}, \mathbf{dyn})$	`pyth_Dyn_Dyn`	`return x*x+y*y`	$\mathbf{dyn}$

If f_{gen} is called with the same $(f, \overline{s})$ again, it looks up the existing entry in the cache instead of re-specialising.

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Shape of Result (sor) — Non-Staged Application

When f is **non-staged**, we don't have its Staged Block. Two sub-cases.

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \forall i.\, \text{static}(s_i) \qquad f_{\text{imp}}\left(\overline{\text{valOf}(s)}\right) = (r, s)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r \Rightarrow s}$$

Example

```
NonStaged.sq(2)                                    mls
```

All args static; evaluate $\text{sq}(2) = 4$.
$\varepsilon \vdash \text{sq}(2) \rightsquigarrow 4 \Rightarrow [\![4]\!]$

$$\frac{f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad \exists i.\, \neg\, \text{static}(s_i)}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow f(\overline{p_i}) \Rightarrow \textbf{dyn}}$$

Example

```
// where x is dynamic                              mls
NonStaged.sq(x)
```

$\Gamma = x \mapsto \textbf{dyn}$; cannot evaluate, call remain unchanged.
$\Gamma \vdash \text{sq}(x) \rightsquigarrow \text{sq}(x) \Rightarrow \textbf{dyn}$

$$r ::= p \mid [\overline{p}] \mid \textbf{new}\, C(\overline{p}) \mid f(\overline{p})$$

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Non-Staged Application Example

```
module NonStaged with                    mls
  fun sq(x) = x * x

staged module Staged with
  fun pyth(x, y) =
    NonStaged.sq(x)
    + NonStaged.sq(y)
  fun test() = pyth(2, 3)
```

After staging:

```
module Staged with                       mls
  fun pyth_Lit2_Lit3() = 13
  fun test() = 13
```

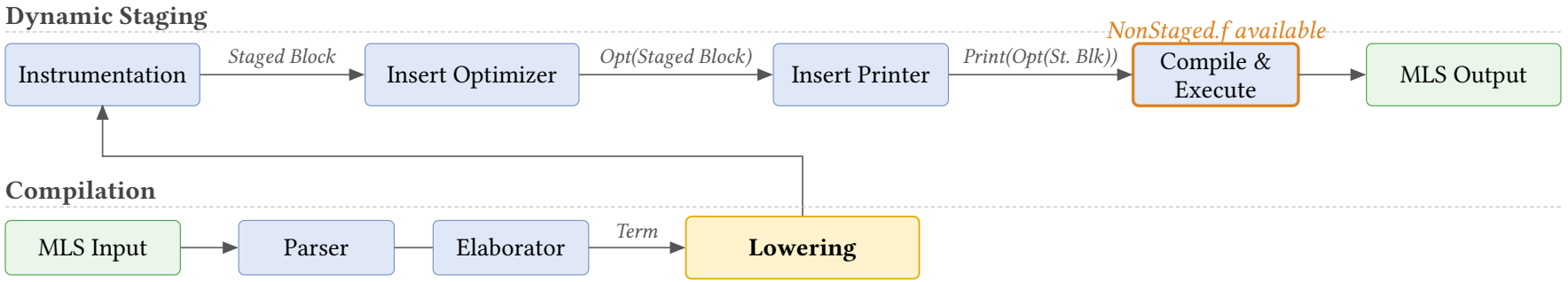
- No specialised functions are generated for the non-staged module `NonStaged`.

Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

How do we call the function `NonStaged.sq`?

Recall that during execution of the optimizer, we already compile the function `NonStaged.sq(x)` to JavaScript.

- Essentially we get the optimization for free in the static parameter case (at no expense of code size)



Shape of Result ($\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

Static Shape

$$\text{static}([\![\iota]\!]) = \textbf{true} \qquad \text{static}([\![[\overline{s_i}^n]]\!]) = \bigwedge_i \text{static}(s_i)$$

$$\text{static}([\![C(\overline{n_i : s_i})]\!]) = \bigwedge_i \text{static}(s_i) \qquad \text{static}(s_1 \cup s_2) = \text{static}(s_1) \wedge \text{static}(s_2)$$

$$\text{static}(\textbf{dyn}) = \textbf{false} \qquad \text{static}(\bot) = \textbf{err}$$

Remark. static(s) returns true iff s only contains one shape (no union) and no subshape of s contains **dyn**.

$$r ::= p \mid [\overline{p}] \mid \textbf{new}\ C(\overline{p}) \mid f(\overline{p})$$

Shape of Block (Pattern Matching)

Pattern Matching

For propagating on Block, the pattern matching Block is the most interesting. Let's dive in with an example.

```mls
class C(val n)
staged module Simple with
  fun f(x) =
    let y
    if x is C then y = x.n
    else y = 0
    y + 1
  fun test()     = f(C(2))
  fun test2(dyn) = f(C(dyn))
  fun test3()    = f(0)
```

Shape of Block (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto$ ⟦`C(n: 2)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto$ ⟦`C(n: 2)`⟧

Shape of Block (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$

Shape of Block (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto [\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto [\![$`C(n: 2)`$]\!]$

`y` $\longmapsto [\![\bot]\!]$

1. `let y`: extend ctx with `y` $\longmapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead

Shape of Block (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$

`y` $\longmapsto$ $[\![$`2`$]\!]$

1. `let y`: extend ctx with `y` $\longmapsto$ $[\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead
3. In `then`: `sop(x.n) = sel(`$[\![$`C(n: 2)`$]\!]$`, n)` = $[\![$`2`$]\!]$, so `y` $\longmapsto$ $[\![$`2`$]\!]$

Shape of Block (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with `x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` $\longmapsto$ $[\![$`C(n: 2)`$]\!]$

`y` $\longmapsto$ $[\![$`2`$]\!]$

1. `let y`: extend ctx with `y` $\longmapsto$ $[\![\bot]\!]$
2. `if x is C`: `filter(`$[\![$`C(n: 2)`$]\!]$`, C)` = $[\![$`C(n: 2)`$]\!]$ so `then` is viable; `rest` = $[\![\bot]\!]$ so `else` is dead
3. In `then`: `sop(x.n) = sel(`$[\![$`C(n: 2)`$]\!]$`, n)` = $[\![$`2`$]\!]$, so `y` $\longmapsto$ $[\![$`2`$]\!]$
4. `y + 1` folds to $[\![$`3`$]\!]$, then `Return 3`

Cached as `f_C_Lit2`; body folds entirely to the constant `3`.

Shape of Block (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦⊥⟧

Same opening as Trace 1 (`let y`, then `if x is C` keeps `then`, kills `else`). Continuing in `then`:

Shape of Block (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                                  mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

Same opening as Trace 1 (`let y`, then `if x is C` keeps `then`, kills `else`). Continuing in `then`:

1. In `then`: `sop(x.n)` = `sel(`⟦`C(n: dyn)`⟧`, n)` = ⟦`dyn`⟧

Shape of Block (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: ⟦`C(n: dyn)`⟧.

Specialise `f` with `x` ⟼ ⟦`C(n: dyn)`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`C(n: dyn)`⟧

`y` ⟼ ⟦`dyn`⟧

Same opening as Trace 1 (`let y`, then `if x is C` keeps `then`, kills `else`). Continuing in `then`:

1. In `then`: `sop(x.n)` = `sel(`⟦`C(n: dyn)`⟧`, n)` = ⟦`dyn`⟧
2. `y + 1`: cannot fold (⟦`dyn`⟧ `+ 1`); emit code, result ⟦`dyn`⟧

Cached as `f_C_Dyn`; body keeps the `y = x.n` assignment.

Shape of Block (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦`0`⟧. Specialise `f` with `x` ⟼ ⟦`0`⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦`0`⟧

Shape of Block (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧. Specialise `f` with x ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

x ⟼ ⟦0⟧

y ⟼ ⟦⊥⟧

1. `let y`: y ⟼ ⟦⊥⟧

Shape of Block (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧. Specialise `f` with `x` ⟼ ⟦0⟧.

```mls
1  fun f(x) =
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦⊥⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable

Shape of Block (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: $[\![0]\!]$. Specialise `f` with $x \mapsto [\![0]\!]$.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

$x \mapsto [\![0]\!]$

$y \mapsto [\![0]\!]$

1. `let y`: $y \mapsto [\![\bot]\!]$
2. `if x is C`: `filter(`$[\![0]\!]$`, C)` = $[\![\bot]\!]$ so `then` is dead; `rest(`$[\![0]\!]$`, C)` = $[\![0]\!]$ so `else` is viable
3. In `else`: $y \mapsto [\![0]\!]$

Shape of Block (Pattern Matching)

Trace 3: `test3() = f(0)`

Argument shape: ⟦0⟧. Specialise `f` with `x` ⟼ ⟦0⟧.

```
1  fun f(x) =                    mls
2    let y
3    if x is C then
4      y = x.n
5    else
6      y = 0
7    y + 1
```

ctx

`x` ⟼ ⟦0⟧

`y` ⟼ ⟦0⟧

1. `let y`: `y` ⟼ ⟦⊥⟧
2. `if x is C`: `filter(`⟦0⟧`, C)` = ⟦⊥⟧ so `then` is dead; `rest(`⟦0⟧`, C)` = ⟦0⟧ so `else` is viable
3. In `else`: `y` ⟼ ⟦0⟧
4. `y + 1` folds to ⟦1⟧, then `Return 1`

Cached as `f_Lit0`.

Shape of Block (Pattern Matching)

Resulting Cache

After all three calls, the staged module contains:

```mls
module Simple with
  fun f_C_Lit2(x) = 3      // Trace A
  fun f_C_Dyn(x) =         // Trace B
    let y
    y = x.n
    y + 1
  fun f_Lit0()    = 1      // Trace C specialised
  fun test()      = 3
  fun test2(dyn)  = f_C_Dyn(C(dyn))
  fun test3()     = 1
```

Pattern Matching (Formalization)

Branch Filter (`filter`)

Intuition. $\text{filter}(s, \pi)$ keeps the part of shape s that **matches** the pattern π — i.e. the shape flowing into a branch.

$$\text{filter}(s, \_) = s$$
$$\text{filter}([\![[\overline{s_i}^n]]\!], []^n) = [\![[\overline{s_i}^n]]\!]$$
$$\text{filter}(\mathbf{dyn}, \pi) = \text{silh}(\pi)$$
$$\text{filter}(s, \pi) = \bot \quad \text{otherwise}$$

$$\text{filter}([\![\iota_1]\!], \iota_2) = [\![\iota_1]\!] \quad \text{if} \quad \iota_1 = \iota_2$$
$$\text{filter}([\![C(\overline{n_i : s_i}^m)]\!], C) = [\![C(\overline{n_i : s_i}^m)]\!]$$
$$\text{filter}(s_1 \cup s_2, \pi) = \text{filter}(s_1, \pi) \cup \text{filter}(s_2, \pi)$$

Example

$$\text{filter}([\![C(n : \mathbf{dyn})]\!], C) = [\![C(n : \mathbf{dyn})]\!]$$
$$\text{filter}([\![0]\!], C) = \bot$$
$$\text{filter}([\![C(n : \mathbf{dyn})]\!] \cup [\![0]\!], C) = \text{filter}([\![C(n : \mathbf{dyn})]\!], C) \cup \text{filter}([\![0]\!], C)$$
$$= [\![C(n : \mathbf{dyn})]\!] \cup \bot = [\![C(n : \mathbf{dyn})]\!]$$

Pattern Matching (Formalization)

Silhouette Function (`silh`)

$\mathrm{silh}(\pi)$ creates the shape of a pattern π, with all sub-shapes set to $\mathbf{dyn}$.

$$\mathrm{silh}(\iota) = [\![\iota]\!] \quad \mathrm{silh}(C) = \left[\!\left[C\left(\overline{\mathbf{dyn}}^{n}\right)\right]\!\right] \quad \mathrm{silh}([]^{n}) = \left[\!\left[\overline{\mathbf{dyn}}^{n}\right]\!\right]$$

Pattern Matching (Formalization)

Branch Remainder (rest)

Intuition. $\mathrm{rest}(s, \pi)$ keeps the part of shape s that does **not** match π — i.e. the shape flowing out of a branch

$$\mathrm{rest}(s, \_) = \bot \qquad \mathrm{rest}(\llbracket \iota_1 \rrbracket, \iota_2) = \bot \quad \text{if} \quad \iota_1 = \iota_2$$

$$\mathrm{rest}(\llbracket [\overline{s_i}^n] \rrbracket, []^n) = \bot \qquad \mathrm{rest}(\llbracket C(\overline{n_i : s_i}^m) \rrbracket, C) = \bot$$

$$\mathrm{rest}(\mathbf{dyn}, \pi) = \mathbf{dyn} \quad \text{if} \quad \pi \neq \_ \qquad \mathrm{rest}(s_1 \cup s_2, \pi) = \mathrm{rest}(s_1, \pi) \cup \mathrm{rest}(s_2, \pi)$$

$$\mathrm{rest}(s, \pi) = s \quad \text{otherwise}$$

Example

$$\begin{aligned} \mathrm{rest}(\llbracket C(n : \mathbf{dyn}) \rrbracket \cup \llbracket 0 \rrbracket, C) &= \mathrm{rest}(\llbracket C(n : \mathbf{dyn}) \rrbracket, C) \cup \mathrm{rest}(\llbracket 0 \rrbracket, C) \\ &= \bot \cup \llbracket 0 \rrbracket = \llbracket 0 \rrbracket \end{aligned}$$

Pattern Matching (Formalization)

Non Termination 1

Consider Fibonacci number:

```mls
staged module Staged with
  fun fib(n) = if n is
    1 then 1
    2 then 1
    n then fib(n - 1) + fib(n - 2)
```

With $\texttt{n} \mapsto [\![\texttt{dyn}]\!]$, every branch is viable. The recursive call `fib(n - 1)` has argument shape $[\![\texttt{dyn}]\!]$, so shape propagation recurses forever, even though the original program terminates for valid inputs.

Pattern Matching (Formalization)

Solution: Stub Insertion

Before propagating into `fib(dyn)`, we **pre-insert a stub** into the cache:

Cache (stub inserted)

Function	Shapes	Symbol	Block	Returned Shape
`fib`	$(\mathbf{dyn},)$	`fib_Dyn`	*stub*	**dyn**

Now when propagating the `fib(n - 1) + fib(n - 2)` arm:

1. `fib(n - 1)` has argument shape $[\![\texttt{dyn}]\!]$ — matches the existing stub $\longrightarrow$ rewritten to `fib_Dyn(n - 1)`.
2. `fib(n - 2)` likewise hits the stub $\longrightarrow$ rewritten to `fib_Dyn(n - 2)`.
3. Result shape of the arm: $[\![\texttt{dyn}]\!] + [\![\texttt{dyn}]\!] = [\![\texttt{dyn}]\!]$.

Pattern Matching (Formalization)

The completed specialised function will be

```mls
fun fib_Dyn(n) = if n is
  1 then 1
  2 then 1
  n then fib_Dyn(n - 1) + fib_Dyn(n - 2)
```

Cache (stub filled)

Function	Shapes	Symbol	Block	Returned Shape
`fib`	$(\mathbf{dyn},)$	`fib_Dyn`	`if n is ...`	**dyn**

Remark. Stub insertion only guarantees termination of shape propagation when **the original program terminates**.

Pattern Matching (Formalization)

Non Termination 2

Consider a function that scans an array for a zero:

```mls
staged module Staged with
  fun f(xs, i) =
    if xs.(i) is
      0 then i
      v then f(xs, i + 1)
```

With `xs` $\mapsto$ ⟦`dyn`⟧ and `i` $\mapsto$ ⟦`0`⟧ on the first call, `xs.(0)` is ⟦`dyn`⟧ — both branches viable.

The recursive call has `i` $\mapsto$ ⟦`1`⟧, then ⟦`2`⟧, ... Shape Propagation diverges :(

Pattern Matching (Formalization)

Solution: Context Decay

Before processing the branches of `if xs.(i) is`, since the scrutinee has shape ⟦`dyn`⟧, we **decay** the context: every path that influenced the scrutinee is refined to ⟦`dyn`⟧.

Here `xs.(i)` depends on `xs` and `i`, so both are decayed to ⟦`dyn`⟧ before entering the branches.

$$\text{decay}(\Gamma, p, s) = \Gamma \quad \text{if} \quad p \neq x \text{ for some variable } x \text{ or } s \neq \mathbf{dyn}$$

$$\text{decay}(\Gamma, x, \mathbf{dyn}) = \Gamma \cdot \left[\overline{p_i \mapsto \mathbf{dyn}}\right]^{p_i \in \text{ alias}(\text{clos}(x))}$$

With `i` $\mapsto$ ⟦`dyn`⟧ after decay, `i + 1` $\mapsto$ ⟦`dyn`⟧ too. The recursive call always hits a stable shape — propagation terminates.

Pattern Matching (Formalization)

Combining `filter`, `rest`, dead branch elimination and `decay` yields the following formalization for pattern matching.

$$\frac{\overline{\Gamma_0 \cdot (p \mapsto s_{i'}) \vdash b_i \rightsquigarrow b_{i'} \Rightarrow \Gamma_i, s_{i''}}^{i=1..n, s_i \neq \bot} \quad \Gamma_0 \bowtie \overline{\Gamma_i - \Gamma_0}^{i=1..n, s_i \neq \bot} \vdash b_r \rightsquigarrow b_{r'} \Rightarrow \Gamma', s \quad \Gamma \vdash p \Rightarrow s_0 \quad \Gamma_0 = \mathrm{decay}(\Gamma, p, s_0) \quad \overline{s_{i'} = \mathrm{filter}(s_{i-1}, \pi_i) \quad s_i = \mathrm{rest}(s_{i-1}, \pi_i)}^{n}}{\Gamma \vdash \mathbf{if}\ p\ \mathbf{is}\ \overline{\pi_i\ \mathbf{then}\ b_i}^{n}; b_r \rightsquigarrow \mathbf{if}\ p\ \mathbf{is}\ \overline{\pi_i\ \mathbf{then}\ b_{i'}}^{i=1..n, s_i \neq \bot}; b_{r'} \Rightarrow s}$$

Staged Class

Similar to staging module, we can stage classes as well, where we specialises on its method, with the main ingredient being how to handle dynamic dispatching `x.f()`

```
1   staged class B1(val y) with
2     fun call(x) = x + 2 + y
3   staged class B2(val y) with
4     fun call(x) = x + y
5   staged module M with
6     fun twice(f, x) = f.call(f.call(x))
7     fun pick(x, y, b) = if b then x else y
8     fun f(b) =
9       let m = pick(new B1(2), new B2(3), b)
10      twice(m, 5)
```

Staged Class

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![$`dyn`$]\!]$.

```
1  fun f(b) =                                  mls
2      let m = pick(new B1(2),new
       B2(3),b)
3      twice(m, 5)
```

f's ctx

`b` $\longmapsto [\![$`dyn`$]\!]$

Staged Class

Trace A: `f(dyn)`

Specialise `f` with `b` $\longmapsto [\![\texttt{dyn}]\!]$.

```
1  fun f(b) =                                   mls
2      let m = pick(new B1(2),new B2(3),b)
3      twice(m, 5)
```

f's ctx

`b` $\longmapsto [\![\texttt{dyn}]\!]$

`m` $\longmapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\longmapsto [\![\bot]\!]$

Staged Class

Trace A: `f(dyn)`

Specialise `f` with `b` $\mapsto [\![\texttt{dyn}]\!]$.

```
1  fun f(b) =                                    mls
2      let m = pick(new B1(2),new B2(3),b)
3    twice(m, 5)
```

f's ctx

`b` $\mapsto [\![\texttt{dyn}]\!]$

`m` $\mapsto [\![\bot]\!]$

1. `let m`: extend ctx with `m` $\mapsto [\![\bot]\!]$
2. Call `pick(new B1(2), new B2(3), b)`: argument shapes are $[\![\texttt{B1(2)}]\!]$, $[\![\texttt{B2(3)}]\!]$, $[\![\texttt{dyn}]\!]$. Recurse into `pick` with a fresh ctx $\rightarrow$ **go to Trace B**

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =                mls
2    if b then x
3    else y
```

pick's ctx

$x \longmapsto [\![\texttt{B1(2)}]\!]$

$y \longmapsto [\![\texttt{B2(3)}]\!]$

$b \longmapsto [\![\texttt{dyn}]\!]$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =                mls
2    if b then x
3    else y
```

pick's ctx

$x \longmapsto [\![\texttt{B1(2)}]\!]$

$y \longmapsto [\![\texttt{B2(3)}]\!]$

$b \longmapsto [\![\texttt{dyn}]\!]$

1. `if b`: `b` is $[\![\texttt{dyn}]\!]$, so both branches viable

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =          mls
2    if b then x
3    else y
```

pick's ctx

$x \longmapsto [\![\texttt{B1(2)}]\!]$

$y \longmapsto [\![\texttt{B2(3)}]\!]$

$b \longmapsto [\![\texttt{dyn}]\!]$

1. `if b`: `b` is $[\![\texttt{dyn}]\!]$, so both branches viable
2. `then` arm returns `x`, shape = $[\![\texttt{B1(2)}]\!]$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =          mls
2    if b then x
3    else y
```

`pick`'s ctx

$\texttt{x} \longmapsto [\![\texttt{B1(2)}]\!]$

$\texttt{y} \longmapsto [\![\texttt{B2(3)}]\!]$

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

1. `if b`: `b` is $[\![\texttt{dyn}]\!]$, so both branches viable
2. `then` arm returns `x`, shape = $[\![\texttt{B1(2)}]\!]$
3. `else` arm returns `y`, shape = $[\![\texttt{B2(3)}]\!]$

Staged Class

Trace B: `pick(B1(2), B2(3), dyn)`

```
1  fun pick(x, y, b) =          mls
2    if b then x
3    else y
```

pick's ctx

$x \longmapsto [\![B1(2)]\!]$

$y \longmapsto [\![B2(3)]\!]$

$b \longmapsto [\![dyn]\!]$

1. `if b`: `b` is $[\![dyn]\!]$, so both branches viable
2. `then` arm returns `x`, shape = $[\![B1(2)]\!]$
3. `else` arm returns `y`, shape = $[\![B2(3)]\!]$
4. Result shape: $[\![B1(2) \cup B2(3)]\!]$. Cached as `pick1`. Return to Trace A.

Staged Class

Trace A (continued): back in `f`

```
1  fun f(b) =                                   mls
2      let m = pick(new B1(2), new B2(3), b)
3    twice(m, 5)
```

f's ctx

b ⟼ ⟦dyn⟧

m ⟼ ⟦B1(2) ∪ B2(3)⟧

Staged Class

Trace A (continued): back in `f`

```
1  fun f(b) =                                   mls
2      let m = pick(new B1(2), new B2(3), b)
3      twice(m, 5)
```

`f`'s ctx

$\texttt{b} \longmapsto [\![\texttt{dyn}]\!]$

$\texttt{m} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

1. Bind `m` to `pick`'s returned shape = $[\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

Staged Class

Trace A (continued): back in `f`

```
1  fun f(b) =                                  mls
2    let m = pick(new B1(2), new
     B2(3), b)
3    twice(m, 5)
```

`f`'s ctx

`b` $\longmapsto$ ⟦`dyn`⟧

`m` $\longmapsto$ ⟦`B1(2)` $\cup$ `B2(3)`⟧

1. Bind `m` to `pick`'s returned shape = ⟦`B1(2)` $\cup$ `B2(3)`⟧
2. `twice(m, 5)`: $\rightarrow$ **go to Trace C** and specialise as `twice1`

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
fun twice(f, x) =
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

twice's ctx

$f \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$x \longmapsto [\![5]\!]$

Recall:

```mls
class B1(y) with
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
1  fun twice(f, x) =
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

Recall:

```mls
1  class B1(y) with
2    fun call(x) = x + 2 + y
3  class B2(y) with
4    fun call(x) = x + y
```

twice's ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\bot]\!]$

1. `let tmp`: extend ctx with $\texttt{tmp} \longmapsto [\![\bot]\!]$

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
fun twice(f, x) =
  let tmp
  tmp = f.call(x)
  f.call(tmp)
```

Recall:

```mls
class B1(y) with
  fun call(x) = x + 2 + y
class B2(y) with
  fun call(x) = x + y
```

twice's ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\bot]\!]$

2. `f.call(x)` on union → **split by class**:
   - `f` = $[\![\texttt{B1(2)}]\!]$:

     B1.call's ctx

     $\texttt{this} \longmapsto [\![\texttt{B1(2)}]\!]$

     $\texttt{x} \longmapsto [\![\texttt{5}]\!]$

     cache `f.call1()` in `B1` → $[\![\texttt{9}]\!]$
   - `f` = $[\![\texttt{B2(3)}]\!]$: cache `f.call1()` in `B2` → $[\![\texttt{8}]\!]$
3. Output a pattern matching → $[\![\texttt{9} \cup \texttt{8}]\!]$

Staged Class

Trace C: `twice({B1(2), B2(3)}, 5)`

```mls
1  fun twice(f, x) =
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

Recall:

```mls
1  class B1(y) with
2    fun call(x) = x + 2 + y
3  class B2(y) with
4    fun call(x) = x + y
```

twice's ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\texttt{9} \cup \texttt{8}]\!]$

4. `tmp = f.call(x)`: bind `tmp` to $[\![\texttt{9} \cup \texttt{8}]\!]$

Staged Class

Trace C (continued): second call

```
1  fun twice(f, x) =                    mls
2    let tmp
3    tmp = f.call(x)
4    f.call(tmp)
```

Recall:

```
1  class B1(y) with                     mls
2    fun call(x) = x + 2 + y
3  class B2(y) with
4    fun call(x) = x + y
```

twice's ctx

$\texttt{f} \longmapsto [\![\texttt{B1(2)} \cup \texttt{B2(3)}]\!]$

$\texttt{x} \longmapsto [\![\texttt{5}]\!]$

$\texttt{tmp} \longmapsto [\![\texttt{9} \cup \texttt{8}]\!]$

5. `f.call(tmp)` with `tmp` in $[\![\texttt{9} \cup \texttt{8}]\!]$: split again, body kept as code
   - `f` = $[\![\texttt{B1}]\!]$: cache `f.call2(x) = x + 2 + 2` in `B1` $\longrightarrow [\![\texttt{13} \cup \texttt{12}]\!]$
   - `f` = $[\![\texttt{B2}]\!]$: cache `f.call2(x) = x + 3` in `B2` $\longrightarrow [\![\texttt{12} \cup \texttt{11}]\!]$
6. Output a pattern matching with a returned shape of $[\![\texttt{13} \cup \texttt{12} \cup \texttt{11}]\!]$

Staged Class

Resulting Cache

After propagation, the residual `M` and the residual classes contain:

```mls
module M with
  fun f(b) =
    let m, tmp1, tmp2
    tmp1 = new B1(2)
    tmp2 = new B2(3)
    m = pick1(tmp1, tmp2, b)
    twice1(m)
  fun pick1(x, y, b) =
    if b then new B1(2) else new B2(3)
```

```mls
fun twice1(f) =
  let tmp
  if f is
    B1 then tmp = f.call1()
    B2 then tmp = f.call1()
  if f is
    B1 then f.call2(tmp)
    B2 then f.call2(tmp)
```

Staged Class

```mls
class B1(y) with
  fun call1() = 9
  fun call2(x) =
    let tmp
    tmp = x + 2
    tmp + 2

class B2(y) with
  fun call1() = 8
  fun call2(x) = x + 3
```

Remark. If `f`'s shape included an **unstaged** class `B3`, we could not specialise the `B3.call` function without its Staged Block. The generated `twice1` would then include an `else` branch falling back to the original `f.call(...)`.

Printing Staged Block

After all the specialised functions are completed, we need to write the functions to a new file. This is a similar to staging, but done in reverse.

We do not export the specialised functions, in order to activate the inliner for the specialised functions.

```mls
staged module Math with
  val funCache = new Map([
    ["pow_Dyn_Lit1", Scoped(...)],
    ["pow_Dyn_Lit2", Scoped(...)],
    ["pow", Scoped(...)],
  ])
```

Printed Output File:

```mls
fun pow_Dyn_Lit1(x) = x
fun pow_Dyn_Lit2(x) =
  let {tmp, tmp1}
  tmp = 1
  tmp1 = pow_Dyn_Lit1(x)
  *(x, tmp1)
module Math with
  fun pow(x, n) = ...
```

Inlining

A typical module after Dynamic Staging would look like the following:

Dynamic Staging Output

```mls
fun pow_Dyn_Lit1(x) =
  let {tmp, tmp1}
  x
fun pow_Dyn_Lit2(x) =
  let {tmp, tmp1}
  tmp = 1
  tmp1 = pow_Dyn_Lit1(x)
  x * tmp1
fun pow_Dyn_Lit3(x) =
let {tmp, tmp1}
tmp = 2
tmp1 = pow_Dyn_Lit2(x)
x * tmp1
module M with
  fun pow(n, x) = ...
  fun cube(x) = ...
```

Inlining

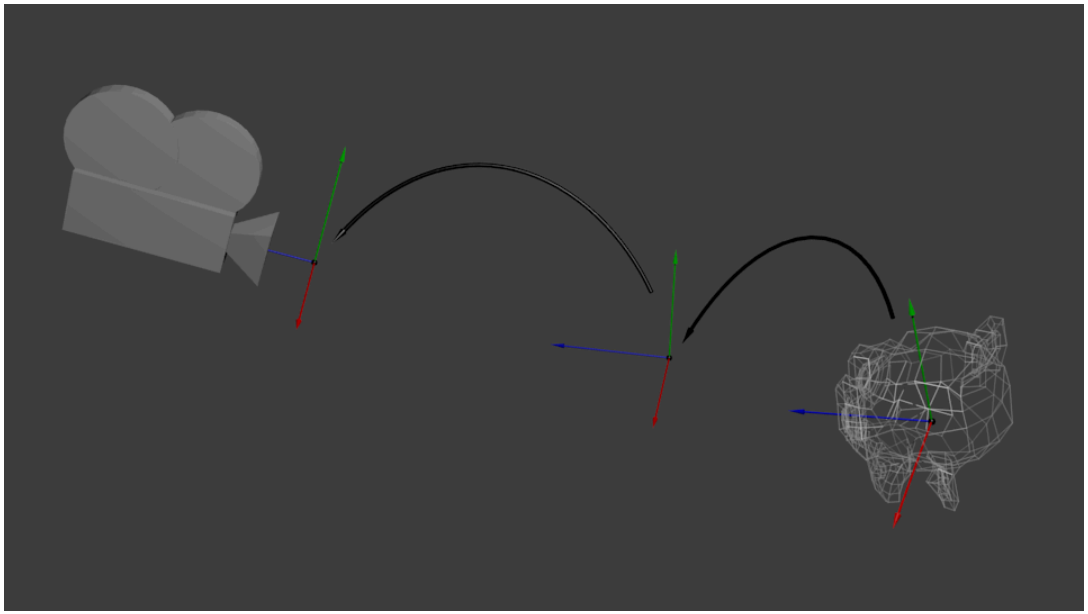
During **the second stage of compilation**, we utilize the MLscript compiler's existing inlining capabilities to inline function calls, as recursion has been eliminated during dynamic staging.

```JavaScript
let x, inlinedVal, tmp1, x1, inlinedVal1, tmp11, x2, inlinedVal2;
x = 2;
x1 = x;
x2 = x1;
inlinedVal2 = x2;
tmp11 = inlinedVal2;
inlinedVal1 = x1 * tmp11;
tmp1 = inlinedVal1;
inlinedVal = x * tmp1;
return inlinedVal
```

Model-View-Projection transformation

Applied in 3D rendering.

$$\vec{v} = MVP \cdot \vec{u}$$



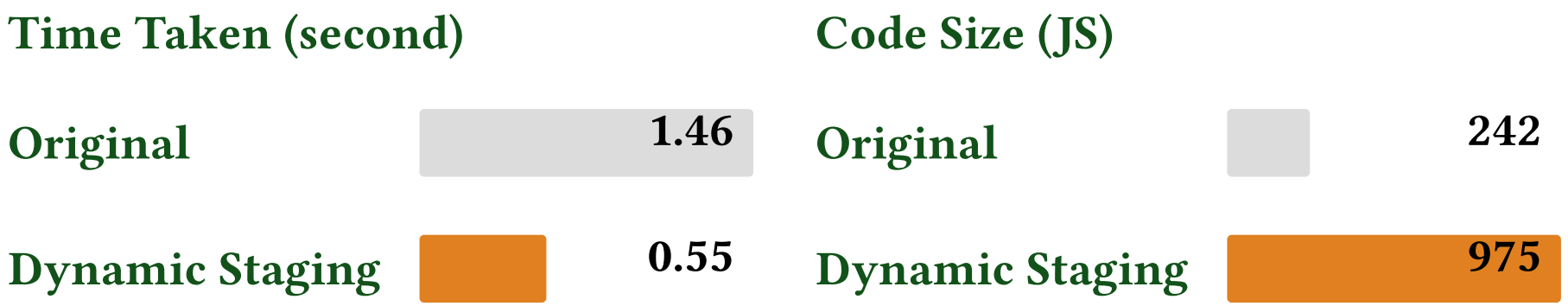
(from opengl-tutorial)

Model-View-Projection transformation

Evaluate matrix multiplications when the matrices are known values.

Benchmark: Transform 16k random coordinates using a known transformation matrices

Environment: MacBook Pro (13-inch, M1, 2020) with 16 GB RAM



Web-demo & QnA

https://chinglongtin.github.io/mlscript/

Bibliography

[1] A. Shali and W. R. Cook, "Hybrid partial evaluation," in *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*, 2011, pp. 375–390.

[2] W. Taha, "A gentle introduction to multi-stage programming," in *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, 2004, pp. 30–50.

[3] K. Swadi, W. Taha, O. Kiselyov, and E. Pasalic, "A monadic approach for avoiding code duplication when staging memoized functions," in *Proceedings of the 2006 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*, 2006, pp. 160–169.